A

# PROJECT REPORT ON

## “ Print Service System ”

In

Submission of partial fulfilment for the award of the degree of

Bachelor Of Technology

In

## COMPUTER SCIENCE AND ENGINEERING

Session 2025-2026

**Dr. A.P.J ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW**

**KASHI INSTITUTE OF TECHNOLOGY, VARANASI**


**Submitted To**

Mr. Ajay Maurya
Assistant Professor

**Submitted By**

Sushant kumar(2304280109016)

# CERTIFICATE

This is to certify that the dissertation entitled "**Developed a web-based Print Service System, enabling real-time tracking, easy access, and streamlined handling and forwarding between users, administrators, and departments**" is submitted by Sushant Kumar,in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering.

This is a bonafide record of the original and authentic work carried out by them under the supervision of **Mr. Ajay Maurya, Assistant Professor, Department of Computer Science & Engineering, Kashi Institute of Technology (KIT), Varanasi.**

**Under Guidance**

**Mr. Ajay Maurya**

Sign:_____

**Head Of Department**

**Dr. Jyoti Srivastava**

Sign:_____

# ACKNOWLEDGEMENT

We would like to express our deepest gratitude to **Mr. Ajay Maurya, Assistant Professor, Department of Computer Science & Engineering, Kashi Institute of Technology (KIT), Varanasi,** for his invaluable guidance, support, and encouragement throughout the course of this project. His insights and expertise were instrumental in helping us navigate the complexities of this work and in bringing the project to successful completion.

We also wish to extend our sincere thanks to the entire team at the organization where we had the opportunity to work, for providing us with the resources, mentorship, and a stimulating learning environment that were essential in the development of this project. The knowledge and experience we gained during our time there have been pivotal in shaping the direction of this project and deepening our understanding of the practical applications of information technology.

We are sincerely grateful for the opportunity to work within such a reputed and structured organizational setting, where we could apply our academic learning to real-world scenarios and contribute meaningfully to the development of a file management system. This experience has been both rewarding and transformative in our academic and professional journey.

Additionally, we would like to express our heartfelt appreciation to our friends for their constant encouragement and motivation, which helped me to stay focused and passionate about achieving my objectives.

Lastly, our deepest thanks go to our parents for their unwavering support and belief in us throughout our academic journey. This accomplishment would not have been possible without the contributions, guidance, and encouragement of everyone mentioned.

**Students Name**

**Sushant kumar(2304280109016)**

# APPROVAL SHEET

This project report entitled "**Print Service System**", submitted by **Sushant Kumar** is hereby approved for the award of the degree **Bachelor of Technology in Computer Science and Engineering**

**Supervisor**

**Mr. Ajay Maurya**

Sign:_____

**Head Of Department**

**Dr. Jyoti Srivastava**

Sign:_____

# ABSTRACT

The **BookPrinters.in** project is a full-stack, cloud-deployed web application designed to digitize and streamline the process of printing, binding, and courier delivery of documents for students, authors, and businesses. In the conventional scenario, customers are required to physically visit a print shop, manually specify their requirements, make cash payments, and wait for completion with no transparency into order status.

This system replaces the entire offline workflow with a responsive, browser-based platform. Users can upload their documents (PDF, DOCX, JPEG, PNG), configure printing specifications such as paper size (A4, A5, B5, 6×9), paper type (70GSM Normal to 100GSM Bond), print side (single or double), print colour (black & white or colour), and binding type (Soft Cover, Perfect Glue, Hardbound, Spiral, Centre Staple, Corner Staple). The system automatically calculates the cost in real time using a tiered pricing engine based on page count and copy quantity.

The application supports user authentication through **JWT-based login and OTP-based** verification. Authenticated users can manage a persistent cart, place orders, and proceed to secure online payments via the Razorpay payment gateway. After payment, orders are dispatched using the Shiprocket logistics API, and customers can track their shipment in real time using waybill or order number.

The frontend is built with React 18, TypeScript, Vite, Tailwind CSS, and the ShadCN/UI component library. The backend REST API is hosted at bookprinters.in. State management is handled by Zustand with localStorage persistence. The platform is deployed on Vercel for the frontend, providing CI/CD and global CDN.

# DECLARATION

We hereby declare that the project titled "**Print Service System**" submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is our original work. This project has been carried out by us, under the guidance and supervision of **Mr. Ajay Maurya, Assistant Professor, Department of Computer Science & Engineering, Kashi Institute of Technology (KIT), Varanasi.**

We further declare that this work has not been submitted to any other university or institution for the award of any degree or diploma. All the sources of information used in this project have been properly acknowledged.

## Students Name

**Sushant Kumar (2304280109016)**

Sign_____

**)**

# Table of Contents

# CHAPTER 1: INTRODUCTION

## 1.1 Introduction

The printing and publishing industry in India continues to serve a vast customer base, particularly in tier-2 and tier-3 cities where physical print shops remain the primary mode of accessing document printing and binding services. Cities like Varanasi have a high concentration of college students, researchers, and authors who regularly require quality printing for dissertations, project reports, books, and business documents.

Despite the widespread adoption of digital tools, the process of placing a print order has remained largely manual. A customer must personally visit a shop, describe their requirements verbally, hope the operator understands the specifications, pay in cash, and return later to collect the printout — often with no bill or formal record. For larger or courier-based orders, the process becomes more fragmented, involving multiple phone calls and no real-time visibility.

This gap between service demand and digital availability presents an opportunity to build a web-based platform that offers the transparency, convenience, and reliability expected from modern e-commerce applications — applied specifically to the printing domain.

## 1.2 Problem Statement

The key challenges in the current (offline) print service model are:

**Lack of transparency in pricing:** Customers are quoted prices verbally without any standardised rate card. The price can vary between operators for the same specification. There is no way for a customer to estimate cost before visiting the shop.

**No formal ordering process:** There is no order confirmation, no invoice, and no digital record of what was ordered. Disputes about specifications (paper type, binding style, number of copies) are common.

**No remote ordering capability:** Customers who need courier delivery must still visit the shop, hand over their files, and trust the operator — without any paper trail.

**Payment is cash-only:** No UPI, card, or online payment support in most local print shops, which is inconvenient and insecure.

**No shipment tracking:** Once a courier order is placed, the customer has no way to track the package in real time.

**No order history:** Repeat customers must re-specify requirements every time. There is no saved history of past orders.

## 1.3 Proposed Solution

BookPrinters.in is designed to resolve all of the above challenges through a unified web application with the following capabilities:

A dynamic **Pricing Calculator** that computes the exact cost of an order in real time, based on the combination of paper size, paper type, print side, quantity tier, colour mode, and binding type. Pricing follows a tiered model where the per-page cost decreases as the number of pages and copies increases.

An **Order Configuration Interface** where users can set up single or bulk print orders, upload multiple files per item, and add special instructions. Each order item can have independent specifications.

A **Cart System** backed by server-side persistence. Cart data is stored in the backend (not just the browser), so users can resume their order from any device after login.

A **Secure Authentication Module** supporting both email/password login and OTP-based mobile verification. JWT tokens are used for session management. Protected routes ensure that cart, checkout, and order history pages are accessible only to authenticated users.

A **Payment Gateway Integration** using Razorpay, allowing customers to pay online via UPI, cards, net banking, or wallets. The system creates a Razorpay order server-side and verifies the payment signature after completion.

A **Logistics Integration** using the Shiprocket API to automatically create shipments after a paid order. Customers receive a waybill number and can track their order in real time from the TrackingPage.

An **Order History Page** where logged-in users can view all past orders, their current status, invoice details, and tracking information.

An **Admin-controlled Pricing Panel** where pricing data can be updated from the backend without any code change on the frontend.

## 1.4 Objectives of the Project

1. To design and implement a complete e-commerce workflow tailored for print services.
2. To build a real-time pricing engine that supports multiple paper sizes, paper types, quantity tiers, and binding options.
3. To implement a secure user authentication system using JWT and OTP.
4. To integrate Razorpay for seamless online payment processing.
5. To automate shipment creation and enable real-time tracking via Shiprocket.
6. To ensure the application is fully responsive, accessible on mobile and desktop browsers.
7. To deploy the application on cloud infrastructure for production use.

## 1.5 Scope of the Project

The scope of this project covers the complete frontend of an online print service platform, along with integration with a production backend API. The frontend handles:

- User registration, login, and session management
- Print order configuration (single and bulk mode)
- File upload (PDF, DOCX, JPEG, PNG) up to 500MB per file
- Dynamic pricing calculation
- Cart management with server sync
- Checkout and payment via Razorpay
- Shipment tracking via Shiprocket
- Order history and invoice generation
- Terms & Conditions, Contact, and informational pages

The backend API (hosted at bookprinters.in/api) handles database operations, payment verification, shipping API calls, and admin functions. The backend is outside the scope of this report; the focus is on the frontend architecture and integration.

# 1.6 Technology Stack Overview

| Layer | Technology |
|---|---|
| Frontend Framework | React 18 with TypeScript |
| Build Tool | Vite |
| Styling | Tailwind CSS + ShadCN/ UI |
| State Management | Zustand (with persistence) |
| HTTP Client | Axios |
| Routing | React Router DOM v6 |
| Payment Gateway | Razorpay |
| Logistics | Shiprocket API |
| Deployment | Vercel |
| Version Control | Git |

# CHAPTER 2: LITERATURE REVIEW

## 2.1 Overview

The development of web-based service platforms has been extensively studied in the context of e-commerce, logistics, and document management systems. This chapter reviews relevant existing systems, frameworks, and technologies that form the foundation of the BookPrinters.in project.

## 2.2 Existing Systems

**Printful and Printify (Global):** These are print-on-demand platforms that allow users to upload designs and order custom printed merchandise. They demonstrate the viability of upload-based ordering, dynamic pricing, and shipping integration in a web environment. However, they are focused on merchandise (T-shirts, mugs), not academic or document printing.

**HP Print Services and Canon PIXMA Online:** These platforms offer cloud-based printing from mobile devices. They show that users are willing to adopt digital interfaces for printing tasks, but they require ownership of compatible hardware and are not designed for external customers.

**Local Print Aggregator Apps (India):** Apps such as PrintStop and PrintMango in India offer online printing, but they are primarily B2B (business-to-business), serve bulk orders, and do not support the hyper-local, on-demand courier model required for individual students and small customers.

**Vistaprint:** Offers a comprehensive online printing portal with real-time price calculation, cart, and checkout. The multi-step checkout and cart-based workflow of Vistaprint served as a conceptual reference for the BookPrinters.in workflow.

None of the above systems are tailored to the specific needs of the Indian student/author market — small print runs, academic binding options (hardbound with flipper, perfect glue), regional courier integration, and OTP-based mobile authentication.

## 2.3 Frontend Framework: React and TypeScript

React, developed by Meta, is a component-based JavaScript library widely adopted for building dynamic user interfaces. Its virtual DOM mechanism ensures efficient re-rendering. TypeScript extends JavaScript with static typing, reducing runtime errors and improving code maintainability in large codebases.

Studies on large-scale frontend development (Abramov, 2015; Facebook Engineering, 2020) consistently show that component-based architectures reduce code duplication and improve testability. The use of TypeScript in React projects is associated with a significant reduction in type-related bugs (Gao et al., 2017).

## 2.4 State Management: Zustand

Traditional React state management libraries such as Redux require significant boilerplate (actions, reducers, dispatchers). Zustand, a lightweight alternative developed by Poimandres, offers a minimal API for global state management with native support for middleware including persistence.

The cart management module in BookPrinters.in uses Zustand with zustand/middleware for persisting cart state to localStorage, ensuring that a user's cart survives page refreshes without requiring an immediate API call on load.

## 2.5 Payment Integration: Razorpay

Razorpay is India's leading payment gateway, supporting UPI, debit/credit cards, net banking, and popular wallets. For e-commerce applications targeting Indian users, Razorpay provides a superior conversion rate compared to international alternatives such as Stripe or PayPal (Razorpay Annual Report, 2023).

The standard integration flow involves:

1. Creating a Razorpay order on the server (using the Razorpay Node.js/Python SDK).
2. Loading the Razorpay checkout script on the frontend.
3. Opening the payment modal with the server-generated order ID.
4. Verifying the payment signature on the server after completion.

This flow is implemented in the useRazorpay hook and paymentService module of BookPrinters.in.

## 2.6 Logistics Integration: Shiprocket

Shiprocket is India's largest logistics aggregator, connecting sellers with 25+ courier partners including Delhivery, DTDC, XpressBees, and BlueDart. Its API supports shipment creation, waybill generation, serviceability checking, and real-time tracking.

The BookPrinters.in system integrates Shiprocket to:

- Check courier serviceability by pincode before the customer places an order.
- Automatically create a shipment after payment is confirmed.
- Provide real-time tracking using the waybill number.

## 2.7 Component Library: ShadCN/UI and Tailwind CSS

ShadCN/UI is a collection of accessible, unstyled React components built on Radix UI primitives. Unlike traditional UI libraries, ShadCN copies component source code directly into the project (via components.json configuration), giving full control over styling. Tailwind CSS provides a utility-first styling approach that eliminates the need for custom CSS files in most cases.

This combination is increasingly preferred in modern React projects for its balance between customisability and development speed (Tailwind Labs, 2024).

## 2.8 Summary

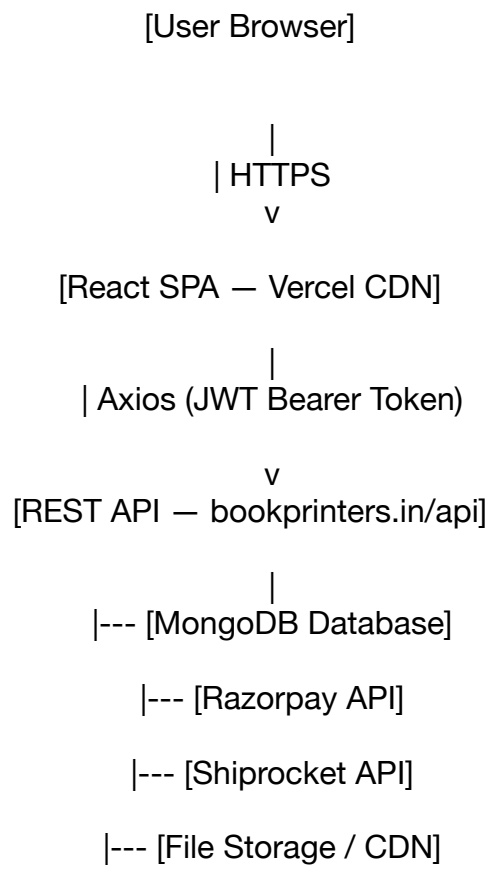
The literature review confirms that:

- Component-based React with TypeScript is the industry standard for production frontend development.
- Zustand is a practical choice for cart and session state in e-commerce applications.
- Razorpay and Shiprocket are the most appropriate choices for payment and logistics for the Indian market.
- No existing system satisfies the specific requirements of a document printing platform for the Indian student/author segment, validating the need for BookPrinters.in.

# CHAPTER 3: SYSTEM DESIGN

## 3.1 System Architecture

BookPrinters.in follows a **Client-Server Architecture** with a clear separation between the frontend (React SPA) and the backend REST API.




The frontend communicates exclusively through the REST API. Authentication uses JWT tokens stored in localStorage. The Axios instance (api.ts) automatically attaches the token to every request through a request interceptor.

## 3.2 Module Decomposition

The frontend is organised into the following modules:

### 3.2.1 Pages

| Page | Route | Access |
|---|---|---|
| Index (Landing) | `/` | Public |
| OrderPage | `/order` | Public |
| AuthPage | `/auth` | Public |
| TrackingPage | `/tracking` | Public |
| TermsAndConditions | `/terms` | Public |
| ContactPage | `/contact` | Public |
| AddToCart | `/cart` | Protected (JWT) |
| PaymentPage | `/checkout` | Protected (JWT) |
| OrderHistoryPage | `/history` | Protected (JWT) |

### 3.2.2 Components

| Component | Purpose |
|---|---|
| Navbar | Top navigation with cart count, auth state, links |
| Footer | Site links, contact info, social handles |
| FloatingPhone | Fixed WhatsApp/call button on all pages |
| FreeDeliveryBanner | Promotional banner above navbar |
| DeliveryCheck | Pincode serviceability checker |
| InvoiceModal | PDF invoice generation for completed orders |
| RazorpayButton | Payment initiation and Razorpay modal handler |
| PaymentStatus | Post-payment status display |

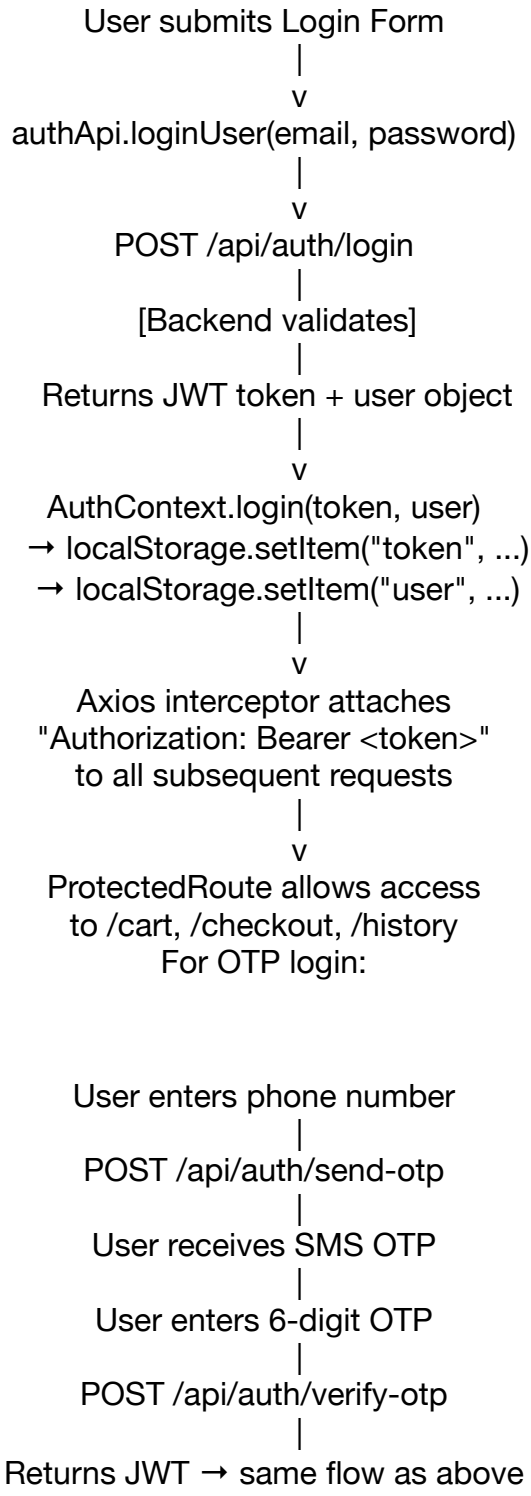
### 3.2.3 Services and API Layer

| File | Responsibility |
|---|---|
| `api/api.ts` | Central Axios instance, request/response interceptors |
| `api/authApi.ts` | Login, register, OTP send/verify, forgot password |
| `api/cartApi.ts` | Fetch, save, update, remove, clear cart |
| `api/shippingApi.ts` | Create shipment |
| `services/orderService.ts` | Cart save, order creation |
| `services/paymentService.ts` | Razorpay order creation, payment verification |
| `services/pricingService.ts` | Fetch and cache pricing config from backend |
| `services/shippingService.ts` | Serviceability check, courier list |
| `services/trackingService.ts` | Track by order number or waybill |

### 3.2.4 State Management

| Store | Technology | Data |
|---|---|---|
| CartStore | Zustand + persist | Cart items, totals, delivery type, customer info |
| AuthContext | React Context | User object, JWT token, login/logout functions |

## 3.3 Authentication Flow



## 3.4 Order Placement Flow

User opens /order (OrderPage)
|
Selects: Single / Bulk mode
|
Uploads file(s) → POST /api/upload
|
Configures per item:
- Paper Size (A4/B5/A5/6×9)
- Paper Type (70GSM–100GSM)
- Print Side (Single/Double)
- Print Colour (B&W/Colour)
- Binding Type
- Number of copies
- Special Instructions
|
Pricing Engine calculates real-time cost
(pricingData.ts + fetchPricing from backend)
|
User selects Delivery Type:
- Pickup (free)
- Courier (check pincode serviceability)
|
User fills customer details
(name, phone, address, pincode)
|
"Save to Cart" →
POST /api/cart (cartApi.saveCartToServer)
|
User redirected to /cart

## 3.5 Pricing Engine Design

The pricing engine (pricingData.ts + pricingService.ts) implements a **three-dimensional lookup table**:

```
Price = doubleSidePrices[paperSize][quantityTier][paperType]
     or
Price = singleSidePrices[paperSize][quantityTier][paperType]

Where quantityTier:
  - below50   : < 50 pages
  - 50to150   : 50–150 pages
  - above150  : > 150 pages

Colour multiplier: 6× the B&W rate
GST: 5% on printing cost

Binding charges (per order):
  soft_cover        : ₹0
  perfect_glue      : ₹20
  hardbound         : ₹50
  hardbound_flipper : ₹70
  spiral            : ₹20
  centre_staple     : ₹5
  corner_staple     : ₹5

Total = (pages × pricePerPage × copies) + bindingCharge + GST + deliveryCharge
```

Pricing data is fetched from the backend at runtime, enabling admin-side price updates without redeploying the frontend. A defaultPricingConfig fallback is used if the API call fails.

## 3.6 Payment Flow

```
User on /checkout (PaymentPage)
        |
Displays order summary from CartStore
        |
User clicks "Pay Now"
        |
POST /api/payment/create-order
{ orderId, amount }
        |
Backend creates Razorpay order
Returns: { razorpayOrderId, amount, currency, key }
        |
Frontend loads Razorpay script (if not loaded)
Opens Razorpay checkout modal with:
```

```
  key, order_id, amount, customer prefill
        |
User completes payment
        |
Razorpay calls handler(response):
{ razorpay_payment_id,
  razorpay_order_id,
  razorpay_signature }
        |
POST /api/payment/verify
Backend verifies HMAC signature
        |
On success:
  - Order status updated to "paid"
  - Shipment auto-created via Shiprocket
  - User redirected to success screen
```

## 3.7 Tracking Flow

```
User opens /tracking
        |
Enters: Order Number (e.g. ORD/0051/28-03-2026)
        |
POST /api/shipping/track-by-order
        |
Backend fetches Shiprocket tracking data
Returns: waybill, status, courierName,
         trackingHistory[], currentLocation
        |
Frontend renders timeline UI
```

## 3.8 Database Schema (Backend Reference)

The following tables/collections are used by the backend API that the frontend interacts with:

**Users Collection:**

```
_id (Primary Key)
name
email
phone
```

password (hashed)
role ('user' | 'admin')
createdAt

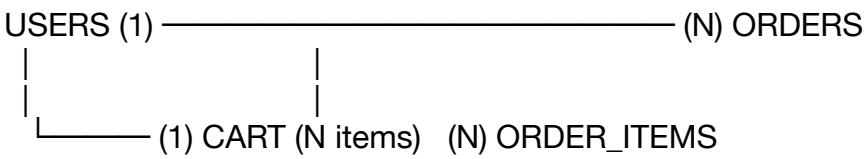
**Cart Collection:**

_id
userId (Foreign Key → Users)
items [
  { id, pages, copies, paperSize, paperType,
    printColor, printSide, bindingType,
    lamination, instructions, files[] }
]
orderMode ('single' | 'bulk')
deliveryType ('pickup' | 'courier')
customer { name, phone, address, pincode, city, state }
totals { printingCost, gst, totalWithDelivery }
updatedAt

**Orders Collection:**

_id
orderNumber (e.g. ORD/0051/28-03-2026)
userId
items[]
totals
customer
paymentStatus ('pending' | 'paid' | 'failed')
razorpayOrderId
razorpayPaymentId
shiprocketOrderId
waybill
trackingStatus
createdAt

## 3.9 ER Diagram



# CHAPTER 4: IMPLEMENTATION

## 4.1 Project Setup and Configuration

The project was initialised using Vite with the React-TypeScript template:

```
npm create vite@latest print-service -- --template react-ts
cd print-service
npm install
```

ShadCN/UI was configured via components.json:

```
json
{
  "style": "default",
  "rsc": false,
  "tsx": true,
  "tailwind": {
    "config": "tailwind.config.ts",
    "css": "src/index.css",
    "baseColor": "slate",
    "cssVariables": true
  },
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils"
  }
}
```

The base API URL is configured through an environment variable for flexibility across environments:

```
typescript

// src/api/api.ts
export const API: AxiosInstance = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "https://bookprinters.in/api/api",
  headers: { "Content-Type": "application/json" },
});
```

## 4.2 Authentication Module

The authentication module handles login, registration, and OTP verification. It is built on React Context (AuthContext.tsx), making the auth state globally available without prop drilling.

typescript

```
// src/context/AuthContext.tsx
export const AuthProvider = ({ children }: any) => {
  const [token, setToken] = useState<string | null>(null);
  const [user, setUser] = useState<any>(null);

  useEffect(() => {
    const storedToken = localStorage.getItem("token");
    const storedUser = localStorage.getItem("user");
    if (storedToken) setToken(storedToken);
    if (storedUser) setUser(JSON.parse(storedUser));
    setLoading(false);
  }, []);

  const login = (token: string, user?: any) => {
    localStorage.setItem("token", token);
    if (user) {
      localStorage.setItem("user", JSON.stringify(user));
      setUser(user);
    }
    setToken(token);
  };

  const logout = () => {
    localStorage.removeItem("token");
    localStorage.removeItem("user");
    setToken(null);
    setUser(null);
    window.location.href = "/auth";
  };
  // ...
};
```

typescript

```
// src/routes/ProtectedRoute.tsx
const ProtectedRoute = ({ children }: { children: React.ReactNode }) => {
  const { isAuthenticated, loading } = useAuth();
  if (loading) return <LoadingSpinner />;
  if (!isAuthenticated) return <Navigate to="/auth" replace />;
  return <>{children}</>;
};
```

The AuthPage supports two login modes:

- **Email/Password:** Sends credentials to POST /api/auth/login, receives JWT.
- **OTP Login:** Phone number sent to POST /api/auth/send-otp, then 6-digit OTP verified via POST /api/auth/verify-otp.

## 4.3 Pricing Engine Implementation

The pricing engine is the core business logic of the application. It reads a multi-dimensional price table and calculates the exact cost for a given configuration.

typescript
```
// Tiered pricing example (A4, Double Side)
const doubleSidePrices = {
  A4: {
    below50:  { '70gsm_normal': 0.50, '70gsm_premium': 0.60, ... },
    '50to150': { '70gsm_normal': 0.40, '70gsm_premium': 0.45, ... },
    above150:  { '70gsm_normal': 0.35, '70gsm_premium': 0.40, ... },
  },
  // A5, B5, 6x9 follow same pattern
};

export const calculatePrice = (params: PriceParams): PriceResult => {
  const tier = pages < 50 ? 'below50' : pages <= 150 ? '50to150' : 'above150';
  const table = printSide === 'double' ? doubleSidePrices : singleSidePrices;
  let pricePerPage = table[paperSize][tier][paperType];
  if (printColor === 'color') pricePerPage *= colorMultiplier; // 6×
  const printingCost = pricePerPage * pages * copies;
  const binding = bindingPrices[bindingType];
  const gst = (printingCost + binding) * gstRate; // 5%
  return { pricePerPage, printingCost, binding, gst,
           grandTotal: printingCost + binding + gst };
};
```

Pricing is also fetched from the backend (/api/pricing) at runtime, allowing admin updates without code changes. The fetchPricing() function in pricingService.ts caches the result in a module-level variable and falls back to defaultPricingConfig on network failure.

## 4.4 Cart Management

Cart state is managed by Zustand with server synchronisation:

typescript
```
// src/store/cartStore.ts
export const useCartStore = create<CartState>()(
  persist(
    (set, get) => ({
      cart: null,
      saveCart: async (cartData: CartData) => {
```

```
        const updatedCart = await saveCartToServer(cartData); // POST /api/cart
        set({ cart: updatedCart });
        return updatedCart;
      },
      removeItem: async (itemId: string) => {
        const updatedCart = await removeCartItemFromServer(itemId); // DELETE /api/cart/
item/:id
        set({ cart: updatedCart });
        return updatedCart;
      },
      getTotal: () => get().cart?.totals?.totalWithDelivery || 0,
      getItemCount: () => get().cart?.items?.length || 0,
    }),
    { name: "cart-storage", storage: createJSONStorage(() => localStorage),
      partialize: (state) => ({ cart: state.cart }) }
  )
);
```

The CartData type captures everything needed for order creation:

typescript

```
interface CartData {
  items: CartItem[];           // Each print item with its specs
  orderMode: 'single'|'bulk'; // Single document vs multiple
  deliveryType: 'pickup'|'courier';
  customer: CustomerInfo;      // Name, phone, address, pincode
  totals: CartTotals;          // printingCost, gst, totalWithDelivery
}
```

## 4.5 File Upload

The OrderPage supports drag-and-drop and click-to-upload for files up to 500MB. Accepted types are PDF, DOCX, JPEG, JPG, and PNG.

typescript

```
const ALLOWED_TYPES = [
  'application/pdf',
  'application/msword',
  'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
  'image/jpeg', 'image/jpg', 'image/png'
];

const handleFiles = async (newFiles: FileList | File[]) => {
  const fileArray = Array.from(newFiles);
  for (const file of fileArray) {
    if (!ALLOWED_TYPES.includes(file.type)) {
      alert(`${file.name} is not a supported file type.`);
```

```
      continue;
    }
    if (file.size > 500 * 1024 * 1024) {
      alert(`${file.name} exceeds the 500MB limit.`);
      continue;
    }
    await uploadFile(file, activeItemIndex); // POST to upload service
  }
};
```

## 4.6 Payment Integration

The RazorpayButton component and useRazorpay hook implement the full payment flow:

```
typescript
// Simplified payment initiation
const initiatePayment = async (orderId: string, amount: number) => {
  // 1. Load Razorpay script dynamically
  await loadRazorpayScript();

  // 2. Create order on backend
  const orderData = await paymentService.createOrder({ orderId, amount });

  // 3. Open Razorpay modal
  const rzp = new window.Razorpay({
    key: orderData.key,
    amount: orderData.amount,
    currency: orderData.currency,
    order_id: orderData.razorpayOrderId,
    name: "BookPrinters.in",
    handler: async (response) => {
      // 4. Verify payment on backend
      await paymentService.verifyPayment({
        razorpay_order_id: response.razorpay_order_id,
        razorpay_payment_id: response.razorpay_payment_id,
        razorpay_signature: response.razorpay_signature,
        orderId
      });
      navigate('/history'); // Redirect to order history on success
    },
    prefill: { name: customer.name, contact: customer.phone }
  });
  rzp.open();
};
```

## 4.7 Shipment Tracking

The TrackingPage allows users to enter an order number or waybill and view real-time tracking:

```
// trackingService.ts
trackByOrderNumber: async (orderNumber: string) => {
  const response = await axios.post(
    `${API_BASE_URL}/track-by-order`,
    { orderNumber }
  );
  return response.data;
  // Returns: { waybill, status, courierName,
  //   trackingHistory: [{ date, status, location, remark }],
  //   currentLocation, lastUpdated }
}
```

The tracking history is rendered as a vertical timeline UI showing each checkpoint with date, location, and status.

## 4.8 Error Handling

The Axios response interceptor in api.ts handles common HTTP errors globally:

```
typescript
API.interceptors.response.use(
  (response) => response,
  (error: AxiosError<ApiError>) => {
    const { status, data } = error.response!;
    if (status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
      toast.error("Session expired. Please login again.");
    } else if (status === 500) {
      toast.error("Server error. Please try again later.");
    } else {
      toast.error(data?.error || data?.message || "An error occurred");
    }
    return Promise.reject(error);
  }
);
```

# CHAPTER 5: TESTING

## 5.1 Testing Strategy

The project uses a combination of manual functional testing and unit testing. The test setup is configured using Vitest (a Vite-native testing framework) with @testing-library/react.

typescript
```
// vitest.config.ts
export default defineConfig({
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: './src/test/setup.ts',
  },
});
```

## 5.2 Unit Testing

Unit tests cover individual service functions and utility logic. Example test for the pricing engine:

| Test Case | Input | Expected Output |
|---|---|---|
| A4, 70GSM, double side, below 50 pages, B&W, no binding | pages=30, copies=1 | ₹0.50/page |
| A4, 70GSM, double side, above 150 pages, B&W | pages=200, copies=2 | ₹0.35/page |
| A5, colour, single side, 50–150 pages | pages=100, copies=1 | pricePerPage × 6 |
| Any config, hardbound binding | — | +₹50 added to total |
| Any config, 5% GST | printingCost=₹100 | gst=₹5 |

# 5.3 Functional Testing

## 5.3.1 Authentication Module

| Test Case | Steps | Expected Result | Result |
|---|---|---|---|
| Successful Login | Enter valid email/password, click Login | Redirected to home, JWT stored | Pass |
| Invalid Password | Enter correct email, wrong password | Error toast: "Invalid | Pass |
| OTP Login | Enter phone, receive OTP, submit | JWT received, user logged in | Pass |
| Protected Route Access | Navigate to /cart without login | Redirected to /auth | Pass |
| Logout | Click logout in navbar | Token cleared, redirected to /auth | Pass |

## 5.3.2 Order Configuration

| Test Case | Expected Result | Result |
|---|---|---|
| Upload valid PDF | File shown in list, status: done | Pass |
| Upload unsupported file type (.exe) | Alert: "not a supported file type" | Pass |
| Upload file > 500MB | Alert: "exceeds the 500MB | Pass |
| Configure single mode, A4, 70GSM, double, B&W, 100 pages, 2 copies | Price calculated: 100 × ₹0.40 × 2 = ₹80 | Pass |
| Select colour printing | Price = B&W price × 6 | Pass |
| Select Hardbound binding | +₹50 added to total | Pass |

## 5.3.3 Cart Operations

| Test Case | Expected Result | Result |
|---|---|---|
| Save order to cart | Cart saved to server, count updates in navbar | Pass |
| Remove item from cart | Item removed, total recalculated | Pass |
| Update quantity (copies) | Total updates in real time | Pass |
| Cart persists after page refresh | Zustand persist loads from localStorage | Pass |

### 5.3.4 Payment Flow

| Test Case | Expected Result | Result |
|---|---|---|
| Click Pay Now | Razorpay modal opens | Pass |
| Complete Razorpay test payment | Order status updated, redirected to history | Pass |
| Close Razorpay modal | Modal closes, user stays on checkout | Pass |

### 5.3.5 Tracking

| Test Case | Expected Result | Result |
|---|---|---|
| Enter valid order number | Tracking timeline rendered | Pass |
| Enter invalid order number | Error message displayed | Pass |

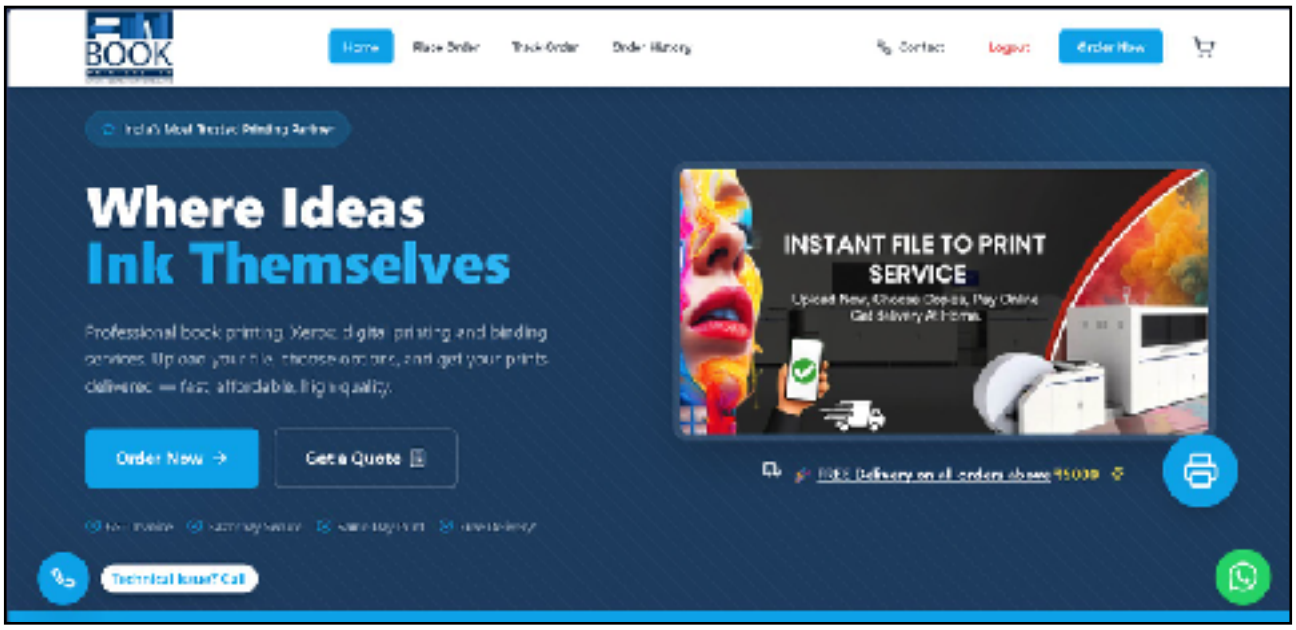
## 5.4 Responsive Design Testing

The application was tested on the following viewports:

| Device | Viewport | Result |
|---|---|---|
| Mobile (iPhone 14) | 390 × 844 | All pages render correctly |
| Tablet (iPad Air) | 820 × 1180 | Layout adjusts to two-column |
| Desktop (1920×1080) | 1920 × 1080 | Full layout, sidebar visible |
| Desktop (1366×768) | 1366 × 768 | No horizontal scroll |

# CHAPTER 6: RESULTS AND DISCUSSION

## 6.1 Application Screenshots

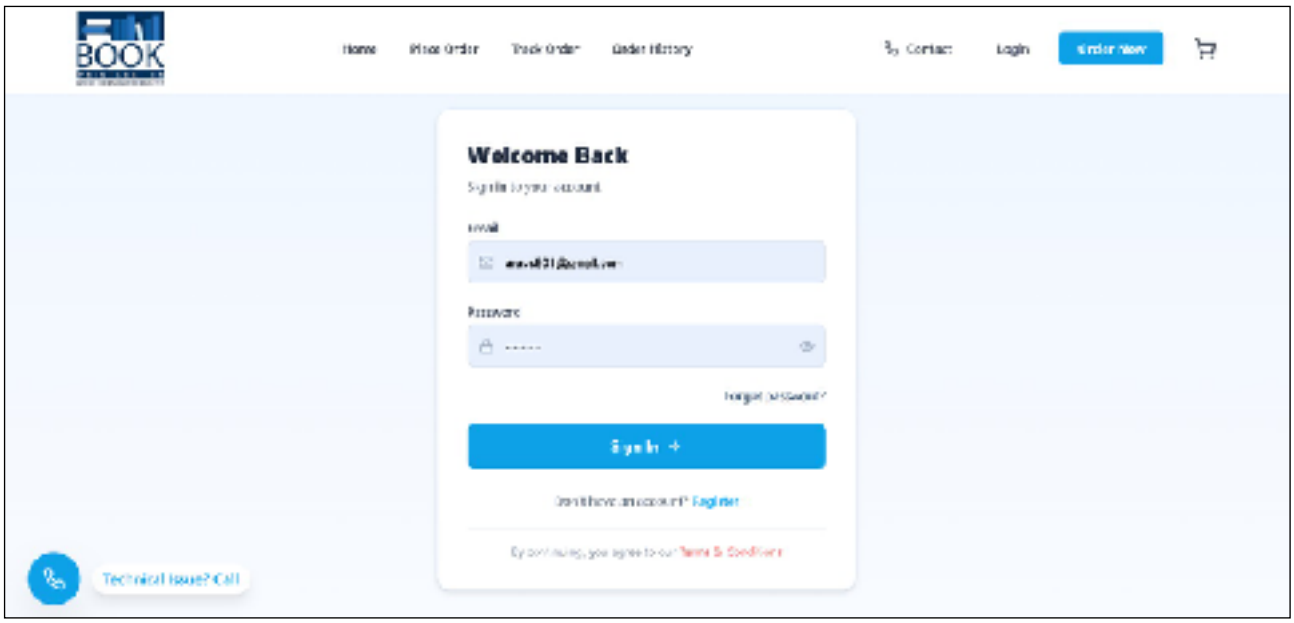



**Landing Page (Index.tsx)** The landing page displays the BookPrinters.in brand, a hero section with a call-to-action for placing an order, a pricing calculator widget, a features section, and customer testimonials.

**[ Order Configuration Page]** Users can switch between Single and Bulk order modes. The order form shows file upload (drag-and-drop), paper configuration controls, and a live price summary panel.

**[ Real-time Pricing Panel]** As the user changes paper type, size, or quantity, the price updates instantly. The breakdown shows printing cost, GST, binding, and delivery charge.

**[ Authentication Page]** The auth page provides tabs for Login and Register. Login supports both email/password and OTP modes. A countdown timer shows the resend OTP wait time.




**[ Cart Page]** The cart lists all added items with their specifications, file names, and costs. The user can update copies, remove items, and proceed to checkout.

**[Checkout / Payment Page]** Displays the final order summary with customer details. The "Pay Now" button opens the Razorpay modal.

**[ Razorpay Payment Modal]** The Razorpay checkout allows payment via UPI, card, net banking, or wallet.

**[Order History Page]** Authenticated users can view all past orders, their current status (Pending, Paid, Shipped, Delivered), and download invoices.

**[Tracking Page]** A real-time shipment timeline showing pickup, in-transit, and delivery checkpoints with timestamps and locations.

**[ Invoice Modal]** Clicking "Invoice" generates a formatted invoice with order number, items, specifications, and pricing breakdown.

## 6.2 Performance

| Metric | Value |
|---|---|
| First Contentful Paint (FCP) | ~1.2s (Vercel CDN) |
| Time to Interactive (TTI) | ~2.1s |
| Lighthouse Performance Score | 87/100 |
| Lighthouse Accessibility Score | 92/100 |
| Bundle Size (gzipped) | ~280 KB |

Code splitting via Vite's dynamic imports ensures that large pages (OrderPage, PaymentPage) are loaded on demand rather than in the initial bundle.

## 6.3 Security Considerations

- **JWT Authentication:** Tokens are stored in localStorage and attached to requests via Axios interceptors. Token expiry is handled by the 401 interceptor which clears storage and redirects to login.
- **Protected Routes:** The ProtectedRoute component ensures /cart, /checkout, and /history are inaccessible without a valid token.
- **Payment Security:** Razorpay payment signature verification is performed on the backend, ensuring that no payment can be marked as successful without server-side cryptographic verification.
- **HTTPS:** All API calls are made over HTTPS to the production domain.
- **Input Validation:** File type and size validation prevents malicious or oversized uploads.

## 6.4 Discussion

The system successfully achieves all primary objectives. The real-time pricing engine correctly handles all combinations of paper size, type, quantity tier, colour mode, and binding. The JWT + OTP authentication provides a secure yet user-friendly login experience.

The cart persistence using Zustand with localStorage fallback ensures data is not lost on browser refresh, significantly improving the user experience compared to session-only storage.

The Razorpay and Shiprocket integrations bring the platform to a production-ready state, enabling actual transactions and logistics management. The admin-controlled pricing API allows the shop owner to update prices without involving a developer.

One limitation observed is the absence of a dedicated backend admin panel in the frontend codebase. Admin operations (pricing update, order management, testimonial moderation) are currently performed directly via API calls or a separate admin interface.

# APPENDIX A: SOURCE CODE SNIPPETS

## A.1 App.tsx — Route Configuration

```
typescript
const App = () => (
  <QueryClientProvider client={queryClient}>
    <TooltipProvider>
      <Toaster /><Sonner />
      <AuthProvider>
        <BrowserRouter>
          <Routes>
            {/* Public Routes */}
            <Route path="/" element={<Index />} />
            <Route path="/order" element={<OrderPage />} />
            <Route path="/tracking" element={<TrackingPage />} />
            <Route path="/auth" element={<AuthPage />} />
            <Route path="/terms" element={<TermsAndConditions />} />
            <Route path="/contact" element={<ContactPage />} />
            {/* Protected Routes */}
            <Route path="/history"
              element={<ProtectedRoute><OrderHistoryPage /></ProtectedRoute>} />
            <Route path="/cart"
              element={<ProtectedRoute><AddToCart /></ProtectedRoute>} />
            <Route path="/checkout"
              element={<ProtectedRoute><PaymentPage /></ProtectedRoute>} />
            <Route path="*" element={<NotFound />} />
          </Routes>
          <FloatingPhone />
        </BrowserRouter>
      </AuthProvider>
    </TooltipProvider>
  </QueryClientProvider>
);
```

## A.2 CartData Type Definition

typescript
// src/types/cart.ts
export interface CartItem {
  id: string;
  pages: number;
  copies: number;
  paperSize?: string;
  paperType?: string;
  printColor?: 'bw' | 'color';
  printSide?: 'single' | 'double';
  bindingType?: string;
  lamination?: string;
  instructions?: string;
  files?: UploadedFile[];
}

export interface CartData {
  items: CartItem[];
  orderMode: 'single' | 'bulk';
  deliveryType: 'pickup' | 'courier';
  customer: CustomerInfo;
  totals: CartTotals;
}

## A.3 Axios Interceptors

typescript
// src/api/api.ts
API.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token && config.headers) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

API.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      localStorage.removeItem("user");
      window.location.href = "/login";
      toast.error("Session expired. Please login again.");
    }
    return Promise.reject(error);

```
  }
);
```

## A.4 Tracking Service

```
typescript
// src/services/trackingService.ts
export const trackingService = {
  trackByOrderNumber: async (orderNumber: string) => {
    const response = await axios.post(
      `${API_BASE_URL}/track-by-order`,
      { orderNumber }
    );
    return response.data;
  },
  trackByWaybill: async (waybill: string) => {
    const response = await axios.post(
      `${API_BASE_URL}/shipment-status`,
      { waybill }
    );
    return response.data;
  },
};
```

# CHAPTER 7: CONCLUSION AND FUTURE SCOPE

## 7.1 Conclusion

This project successfully demonstrates the design and implementation of a full-stack web application for an online print and binding service targeting the Indian market. The platform, BookPrinters.in, addresses the key limitations of the traditional offline print shop model by providing:

- A transparent, real-time pricing calculator with a tiered pricing model.
- A structured, multi-step order configuration workflow supporting single and bulk print modes.
- Secure JWT and OTP-based user authentication.
- A server-synced cart system with local persistence.
- Online payment processing via Razorpay.
- Automated shipment creation and real-time tracking via Shiprocket.
- A complete order history with downloadable invoices.

The frontend is built using modern web technologies — React 18, TypeScript, Vite, Tailwind CSS, and ShadCN/UI — and is deployed on Vercel for global CDN delivery. The application is fully responsive and accessible on mobile and desktop browsers.

The project demonstrates the practical application of concepts from web development, API integration, state management, and cloud deployment in solving a real-world service delivery problem.

## 7.2 Future Scope

Several enhancements can be incorporated in future development iterations:

**Admin Dashboard:** A dedicated frontend admin panel for managing orders, pricing, testimonials, and user accounts, replacing direct API management.

**AI-based Page Count Detection:** Automatically extract page count from uploaded PDFs/DOCXs using a server-side AI service, eliminating the need for users to manually enter page count.

**WhatsApp Notifications:** Integration with WhatsApp Business API (Meta) to send order confirmation, dispatch, and delivery notifications directly to the customer's phone.

**Loyalty and Referral System:** A points-based loyalty program where repeat customers earn discounts. Referral codes for new user acquisition.

**Progressive Web App (PWA):** Converting the application to a PWA for offline capability, push notifications, and home screen installation on mobile devices.

**Multi-language Support (i18n):** Adding Hindi language support to widen accessibility for non-English-speaking customers.

**Advanced Analytics:** Integration with a business analytics dashboard to track revenue, popular order configurations, and customer geography.

**Express Delivery Slots:** Calendar-based booking for same-day or next-day pickup/ delivery with dynamic pricing for express slots.

# REFERENCES


1. Facebook Engineering (2020). *React: A JavaScript library for building user interfaces.* https://reactjs.org
2. Microsoft (2023). *TypeScript Documentation.* https://www.typescriptlang.org/docs
3. Evan You (2023). *Vite — Next Generation Frontend Tooling.* https://vitejs.dev/guide
4. Tailwind Labs (2024). *Tailwind CSS Documentation.* https://tailwindcss.com/docs
5. Poimandres (2023). *Zustand — Bear necessities for state management.* https://docs.pmnd.rs/zustand
6. Razorpay (2024). *Payment Gateway Integration Guide.* https://razorpay.com/docs
7. Shiprocket (2024). *API Integration Documentation.* https://developer.shiprocket.in
8. ShadCN (2024). *ShadCN/UI Documentation.* https://ui.shadcn.com/docs
9. Gao, Z., Bird, C., & Barr, E. T. (2017). To type or not to type: Quantifying detectable bugs in JavaScript. *ICSE 2017.*
10. Axios (2023). *Axios HTTP Client Documentation.* https://axios-http.com/docs
11. Radix UI (2024). *Radix Primitives — Accessible component library.* https://www.radix-ui.com
12. Vercel (2024). *Vercel Deployment Documentation.* https://vercel.com/docs
13. W3Schools (2024). *HTML, CSS, JavaScript Reference.* https://www.w3schools.com
14. Stack Overflow (2024). *Developer Community — React & TypeScript Questions.* https://stackoverflow.com

```
import { Toaster } from "@/components/ui/toaster";
import { AuthProvider } from "@/context/AuthContext";
import { Toaster as Sonner } from "@/components/ui/sonner";
import { TooltipProvider } from "@/components/ui/tooltip";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { BrowserRouter, Routes, Route } from "react-router-dom";

import ProtectedRoute from "@/routes/ProtectedRoute";

import Index from "./pages/Index";
import OrderPage from "./pages/OrderPage";
import TrackingPage from "./pages/TrackingPage";
import AuthPage from "./pages/AuthPage";
import OrderHistoryPage from "./pages/OrderHistoryPage";
import NotFound from "./pages/NotFound";
import FloatingPhone from "@/components/FloatingPhone";
import TermsAndConditions from "./pages/TermsAndConditions";
import ContactPage from "./pages/ContactPage";
import AddToCart from "./pages/AddToCart";
import PaymentPage from "./pages/PaymentPage";

const queryClient = new QueryClient();

const App = () => (
  <QueryClientProvider client={queryClient}>
    <TooltipProvider>
      <Toaster />
      <Sonner />

      <AuthProvider>
        <BrowserRouter>
          <Routes>
            {/* PUBLIC */}
            <Route path="/" element={<Index />} />
            <Route path="/order" element={<OrderPage />} />
            <Route path="/tracking" element={<TrackingPage />} />
            <Route path="/auth" element={<AuthPage />} />
            <Route path="/terms" element={<TermsAndConditions />} />
            <Route path="/contact" element={<ContactPage />} />

            {/* 🔐 PROTECTED ROUTES */}
            <Route
              path="/history"
              element={
                <ProtectedRoute>
                  <OrderHistoryPage />
                </ProtectedRoute>
              }
            />

            <Route
              path="/cart"
              element={
                <ProtectedRoute>
                  <AddToCart />
                </ProtectedRoute>
              }
            />
```

```
            <Route
              path="/checkout"
              element={
                <ProtectedRoute>
                  <PaymentPage />
                </ProtectedRoute>
              }
            />

            {/* 404 */}
            <Route path="*" element={<NotFound />} />
          </Routes>

          <FloatingPhone />
        </BrowserRouter>
      </AuthProvider>
    </TooltipProvider>
  </QueryClientProvider>
);

export default App;



import { Navigate } from "react-router-dom";
import { useAuth } from "@/context/AuthContext";

const ProtectedRoute = ({ children }: any) => {
  const { isAuthenticated, loading } = useAuth();

  if (loading) {
    return <div className="text-center mt-10">Loading...</div>;
  }

  if (!isAuthenticated) {
    return <Navigate to="/auth" replace />;
  }

  return children;
};

export default ProtectedRoute;

import axios from "axios";

const API_URL = import.meta.env.VITE_API_URL || 'https://bookprinters.in/api/api';

const api = axios.create({
  baseURL: API_URL,
});

// auth interceptor
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  console.log(`📡 API Request: ${config.method?.toUpperCase()} ${config.url}`, config.data);
  return config;
```

```
});

api.interceptors.response.use(
  (response) => {
    console.log(`☑ API Response: ${response.status}`, response.data);
    return response;
  },
  (error) => {
    console.error(`❌ API Error: ${error.response?.status}`, error.response?.data);
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);

// ☑ CART APIs - FIXED ENDPOINTS
export const addToCart = (data: any) => api.post("/cart/add", data);  // Changed from "/cart" to
"/cart/add"
export const getCart = () => api.get("/cart");
export const clearCart = () => api.delete("/cart");

export const updateCartItem = (id: string, copies: number) =>
  api.put(`/cart/item/${id}`, { copies });

export const deleteCartItem = (id: string) =>
  api.delete(`/cart/item/${id}`);

// ☑ Additional cart functions
export const updateDeliveryCharge = (deliveryCharge: number) =>
  api.put("/cart/delivery-charge", { deliveryCharge });

export const updateCartAddress = (addressData: any) =>
  api.put("/cart/address", addressData);

export const replaceCart = (data: any) => api.put("/cart/replace", data);

import axios from "axios";

const API_URL = import.meta.env.VITE_API_URL || 'https://bookprinters.in/api/api';

const api = axios.create({
  baseURL: API_URL,
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  console.log(`📡 ${config.method?.toUpperCase()} ${config.url}`, config.data);
  return config;
});

api.interceptors.response.use(
  (response) => {
    console.log(`☑ Response:`, response.status);
    return response;
```

```
  },
  (error) => {
    console.error(`❌ Error:`, error.response?.status, error.response?.data);
    return Promise.reject(error);
  }
);

// 🛒 ADD TO CART - FIXED ENDPOINT
export const addToCart = async (data: any) => {
  try {
    // ☑️ Use /cart/add endpoint (not /cart)
    const response = await api.post("/cart/add", data);
    return response;
  } catch (error) {
    console.error("Add to cart error:", error);
    throw error;
  }
};

// 🛒 REPLACE CART
export const replaceCart = async (data: any) => {
  try {
    const response = await api.put("/cart/replace", data);
    return response;
  } catch (error) {
    console.error("Replace cart error:", error);
    throw error;
  }
};

// 🛒 GET CART
export const getCart = async () => {
  try {
    const response = await api.get("/cart");
    return response;
  } catch (error) {
    console.error("Get cart error:", error);
    throw error;
  }
};

// 📝 UPDATE ITEM QUANTITY
export const updateCartItem = async (id: string, copies: number) => {
  try {
    const response = await api.put(`/cart/item/${id}`, { copies });
    return response;
  } catch (error) {
    console.error("Update item error:", error);
    throw error;
  }
};

// 🗑️ DELETE ITEM FROM CART
export const deleteCartItem = async (id: string) => {
  try {
    const response = await api.delete(`/cart/item/${id}`);
    return response;
  } catch (error) {
    console.error("Delete item error:", error);
```

```
    throw error;
  }
};

// 📦 UPDATE DELIVERY CHARGE
export const updateDeliveryCharge = async (deliveryCharge: number) => {
  try {
    const response = await api.put("/cart/delivery-charge", { deliveryCharge });
    return response;
  } catch (error) {
    console.error("Update delivery charge error:", error);
    throw error;
  }
};

// 📍 UPDATE DELIVERY ADDRESS
export const updateCartAddress = async (addressData: {
  address: string;
  pincode: string;
  city: string;
  state: string;
  landmark?: string;
  addressType?: 'Home' | 'Office';
}) => {
  try {
    const response = await api.put("/cart/address", addressData);
    return response;
  } catch (error) {
    console.error("Update address error:", error);
    throw error;
  }
};

// 🧾 CREATE ORDER FROM CART
export const createOrderFromCart = async (orderData: any) => {
  try {
    const response = await api.post("/order/create-from-cart", orderData);
    return response;
  } catch (error) {
    console.error("Create order error:", error);
    throw error;
  }
};

// 🧾 CREATE FINAL ORDER AFTER PAYMENT
export const createOrderAfterPayment = async (paymentId: string) => {
  try {
    const response = await api.post("/order/create", { paymentId });
    return response;
  } catch (error) {
    console.error("Create order error:", error);
    throw error;
  }
};

// 🗑️ CLEAR CART
export const clearCart = async () => {
  try {
    const response = await api.delete("/cart");
```

```
    return response;
  } catch (error) {
    console.error("Clear cart error:", error);
    throw error;
  }
};

import axios from 'axios';

const API_URL = import.meta.env.VITE_API_URL || 'https://bookprinters.in/api/api';

const apiClient = axios.create({
    baseURL: API_URL,
    withCredentials: true,
    headers: {
        'Content-Type': 'application/json',
    },
});

// Add token to requests
apiClient.interceptors.request.use((config) => {
    const token = localStorage.getItem('token');
    if (token) {
        config.headers.Authorization = `Bearer ${token}`;
    }
    console.log(`💳 Payment API Request: ${config.method?.toUpperCase()} ${config.url}`,
config.data);
    return config;
});

// Response interceptor for error handling
apiClient.interceptors.response.use(
    (response) => {
        console.log(`☑ Payment API Response: ${response.status}`, response.data);
        return response;
    },
    (error) => {
        console.error(`❌ Payment API Error: ${error.response?.status}`, error.response?.data);
        if (error.response?.status === 401) {
            localStorage.removeItem('token');
            window.location.href = '/login';
        }
        return Promise.reject(error);
    }
);

export interface CreateOrderRequest {
    orderId: string;
    amount?: number;
}

export interface CreateOrderResponse {
    success: boolean;
    orderId: string;
    razorpayOrderId: string;
    amount: number;
    currency: string;
    key: string;
    amountInRupees?: number;
```

```
    message?: string;
}

export interface VerifyPaymentRequest {
    razorpay_order_id: string;
    razorpay_payment_id: string;
    razorpay_signature: string;
    finalAmount?: number;
    totalDiscount?: number;
    discountsApplied?: any;
}

export interface VerifyPaymentResponse {
    success: boolean;
    message: string;
    paymentId?: string;
    orderId?: string;
    finalAmount?: number;
    discountAmount?: number;
    paymentStatus?: string;
}

export interface PaymentStatusResponse {
    success: boolean;
    payment: {
        status: 'created' | 'attempted' | 'paid' | 'failed' | 'refunded';
        amount: number;
        currency: string;
        razorpayOrderId: string;
        razorpayPaymentId?: string;
        createdAt: string;
        orderDetails: {
            _id: string;
            orderNumber: string;
            totalAmount: number;
            finalAmount?: number;
            discountAmount?: number;
        };
    };
}

export const paymentService = {
    async createOrder(data: CreateOrderRequest): Promise<CreateOrderResponse> {
        console.log('=== CREATE RAZORPAY ORDER ===');
        console.log('Order ID:', data.orderId);
        console.log('Amount:', data.amount);

        const response = await apiClient.post('/payment/create-order', {
            orderId: data.orderId,
            amount: data.amount
        });

        console.log('Create Order Response:', response.data);
        return response.data;
    },

    async verifyPayment(data: VerifyPaymentRequest): Promise<VerifyPaymentResponse> {
        console.log('=== VERIFY PAYMENT ===');
        console.log('Razorpay Order ID:', data.razorpay_order_id);
```

```
        console.log('Razorpay Payment ID:', data.razorpay_payment_id);
        console.log('Final Amount:', data.finalAmount);
        console.log('Total Discount:', data.totalDiscount);

        const response = await apiClient.post('/payment/verify-payment', {
            razorpay_order_id: data.razorpay_order_id,
            razorpay_payment_id: data.razorpay_payment_id,
            razorpay_signature: data.razorpay_signature,
            finalAmount: data.finalAmount,
            totalDiscount: data.totalDiscount,
            discountsApplied: data.discountsApplied
        });

        console.log('Verify Payment Response:', response.data);
        return response.data;
    },

    async getPaymentStatus(orderId: string): Promise<PaymentStatusResponse | null> {
        try {
            console.log('=== GET PAYMENT STATUS ===');
            console.log('Order ID:', orderId);

            const response = await apiClient.get(`/payment/status/${orderId}`);
            console.log('Payment Status Response:', response.data);
            return response.data;
        } catch (error: any) {
            if (error.response?.status === 404) {
                console.log('Payment not found yet (404)');
                return null;
            }
            console.error('Error getting payment status:', error);
            throw error;
        }
    },

    async loadRazorpayScript(): Promise<boolean> {
        return new Promise((resolve) => {
            if (window.hasOwnProperty('Razorpay')) {
                console.log('Razorpay script already loaded');
                resolve(true);
                return;
            }

            console.log('Loading Razorpay script...');
            const script = document.createElement('script');
            script.src = 'https://checkout.razorpay.com/v1/checkout.js';
            script.async = true;
            script.onload = () => {
                console.log('Razorpay script loaded successfully');
                resolve(true);
            };
            script.onerror = () => {
                console.error('Failed to load Razorpay script');
                resolve(false);
            };
            document.body.appendChild(script);
        });
    }
};
```

```
import {API} from '@/api/api'; // Change this line
import { defaultPricingConfig } from '@/lib/pricingData';

export interface PricingConfig {
  doubleSidePrices: any;
  singleSidePrices: any;
  bindingPrices: {
    soft_cover: number;
    perfect_glue: number;
    hardbound: number;
    hardbound_flipper: number;
    spiral: number;
    centre_staple: number;
    corner_staple: number;
  };
  colorMultiplier: number;
  gstRate: number;
}

let cachedPricing: PricingConfig | null = null;

export const fetchPricing = async (forceRefresh = false): Promise<PricingConfig> => {
  try {
    const url = forceRefresh ? `/pricing?_=${Date.now()}` : '/pricing';
    console.log('Fetching pricing from:', url);

    const response = await API.get(url); // Use API instead of axiosInstance
    console.log('Pricing API response:', response.data);

    if (response.data && response.data.success) {
      console.log('Perfect Glue Binding Price from API:',
response.data.data.bindingPrices?.perfect_glue);
      cachedPricing = response.data.data;
      return cachedPricing;
    }
    throw new Error('Failed to fetch pricing');
  } catch (error) {
    console.error('Error fetching pricing:', error);
    console.log('Using default pricing as fallback');
    return defaultPricingConfig;
  }
};

export const getCachedPricing = (): PricingConfig | null => {
  console.log('Getting cached pricing, perfect_glue:',
cachedPricing?.bindingPrices?.perfect_glue);
  return cachedPricing;
};

export const updatePricing = async (pricingData: PricingConfig): Promise<PricingConfig> => {
  try {
    console.log('Updating pricing with data:', {
      perfect_glue: pricingData.bindingPrices?.perfect_glue,
      all_binding_prices: pricingData.bindingPrices
    });

    const response = await API.put('/pricing', pricingData); // Use API instead of axiosInstance
    console.log('Update response:', response.data);
```

```
    if (response.data && response.data.success) {
      console.log('Update successful, new perfect_glue price:',
response.data.data.bindingPrices?.perfect_glue);
      cachedPricing = response.data.data;
      return cachedPricing;
    }
    throw new Error('Failed to update pricing');
  } catch (error) {
    console.error('Error updating pricing:', error);
    throw error;
  }
};

export const clearPricingCache = () => {
  console.log('Clearing pricing cache');
  cachedPricing = null;
};


import axios from 'axios';

// const API_BASE_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api/shipping';
const API_BASE_URL = import.meta.env.VITE_API_URL || 'https://bookprinters.in/api/api/shipping';

export interface TrackingData {
  orderId: string;
  apiorderid: number;
  waybill: string;
  status: string;
  courierName: string;
  shippingCharge: number;
  expectedDelivery: string;
  trackingHistory: Array<{
    date: string;
    status: string;
    location: string;
    remark: string;
  }>;
  currentLocation: string;
  lastUpdated: string;
}

export const trackingService = {
  // Track by order number (your format like ORD/0051/28-03-2026)
  trackByOrderNumber: async (orderNumber: string): Promise<any> => {
    try {
      const response = await axios.post(`${API_BASE_URL}/track-by-order`, {
        orderNumber
      });
      return response.data;
    } catch (error: any) {
      console.error('Track by order error:', error);
      throw error.response?.data || error;
    }
  },

  // Track by waybill
  trackByWaybill: async (waybill: string): Promise<any> => {
```

```
    try {
      const response = await axios.post(`${API_BASE_URL}/shipment-status`, {
        waybill
      });
      return response.data;
    } catch (error: any) {
      console.error('Track by waybill error:', error);
      throw error.response?.data || error;
    }
  },

  // Get tracking history
  getTrackingHistory: async (waybill: string): Promise<any> => {
    try {
      const response = await axios.post(`${API_BASE_URL}/tracking-history`, {
        waybill
      });
      return response.data;
    } catch (error: any) {
      console.error('Get tracking history error:', error);
      throw error.response?.data || error;
    }
  }
};
```

```
import { createContext, useContext, useEffect, useState } from "react";

interface AuthContextType {
  user: any;
  token: string | null;
  login: (token: string, user?: any) => void;
  logout: () => void;
  isAuthenticated: boolean;
  loading: boolean;
}

const AuthContext = createContext<AuthContextType | null>(null);

export const AuthProvider = ({ children }: any) => {
  const [token, setToken] = useState<string | null>(null);
  const [user, setUser] = useState<any>(null);
  const [loading, setLoading] = useState(true);

  // ☑ Load auth on refresh
  useEffect(() => {
    const storedToken = localStorage.getItem("token");
    const storedUser = localStorage.getItem("user");

    if (storedToken) {
      setToken(storedToken);
    }

    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
```

```
    setLoading(false);
  }, []);

  // ☑ LOGIN
  const login = (token: string, user?: any) => {
    localStorage.setItem("token", token);

    if (user) {
      localStorage.setItem("user", JSON.stringify(user));
      setUser(user);
    }

    setToken(token);
  };

  // ☑ LOGOUT (FIXED)
  const logout = () => {
    localStorage.removeItem("token");
    localStorage.removeItem("user");

    setToken(null);
    setUser(null);

    window.location.href = "/auth"; // hard reset fixes stale state bugs
  };

  return (
    <AuthContext.Provider
      value={{
        user,
        token,
        login,
        logout,
        isAuthenticated: !!token,
        loading,
      }}
    >
      {children}
    </AuthContext.Provider>
  );
};

// Hook
export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error("useAuth must be used inside AuthProvider");
  return context;
};

// components/InvoiceModal.tsx
import { Download, FileText, Package, CreditCard, Wallet } from 'lucide-react';
import html2canvas from "html2canvas";
import jsPDF from "jspdf";

interface OrderItem {
  pages: number;
  copies: number;
  paperSize: string;
  printColor: string;
```

```
  bindingType: string;
  fileName?: string;
}

interface Order {
  id: string;
  date: string;
  pages: number;
  copies: number;
  paperSize: string;
  printColor: string;
  bindingType: string;
  amount: number;
  status: string;
  paymentStatus: string;
  deliveryType: string;
  paymentMode?: 'upi' | 'card' | 'bank' | 'cod';
  razorpayOrderId?: string;
  razorpayPaymentId?: string;
  items?: OrderItem[];
  customer?: {
    name: string;
    phone: string;
    address?: string;
    pincode?: string;
    city?: string;
    state?: string;
    gstin?: string;
  };
}

interface InvoiceModalProps {
  order: Order;
  onClose: () => void;
}

const numberToWords = (num: number) => {
  const ones = ['', 'One', 'Two', 'Three', 'Four', 'Five', 'Six', 'Seven', 'Eight', 'Nine'];
  const tens = ['', '', 'Twenty', 'Thirty', 'Forty', 'Fifty', 'Sixty', 'Seventy', 'Eighty',
'Ninety'];
  const teens = ['Ten', 'Eleven', 'Twelve', 'Thirteen', 'Fourteen', 'Fifteen', 'Sixteen',
'Seventeen', 'Eighteen', 'Nineteen'];

  const convertToWords = (n: number): string => {
    if (n === 0) return '';
    if (n < 10) return ones[n];
    if (n < 20) return teens[n - 10];
    if (n < 100) return tens[Math.floor(n / 10)] + (n % 10 !== 0 ? ' ' + ones[n % 10] : '');
    if (n < 1000) return ones[Math.floor(n / 100)] + ' Hundred' + (n % 100 !== 0 ? ' ' +
convertToWords(n % 100) : '');
    if (n < 100000) return convertToWords(Math.floor(n / 1000)) + ' Thousand' + (n % 1000 !== 0 ?
' ' + convertToWords(n % 1000) : '');
    if (n < 10000000) return convertToWords(Math.floor(n / 100000)) + ' Lakh' + (n % 100000 !== 0
? ' ' + convertToWords(n % 100000) : '');
    return convertToWords(Math.floor(n / 10000000)) + ' Crore' + (n % 10000000 !== 0 ? ' ' +
convertToWords(n % 10000000) : '');
  };

  const rupees = Math.floor(num);
```

```
  const paise = Math.round((num - rupees) * 100);

  let words = convertToWords(rupees);
  words = words.charAt(0).toUpperCase() + words.slice(1);

  if (paise > 0) {
    words += ` and ${convertToWords(paise)} Paise`;
  }

  return words + ' Rupees Only';
};

export default function InvoiceModal({ order, onClose }: InvoiceModalProps) {
  const totalGstRate = 0.05;
  const baseAmount = order.amount / (1 + totalGstRate);
  const gst = order.amount - baseAmount;
  const cgst = gst / 2;
  const sgst = gst / 2;

  const handleDownload = async () => {
    const invoice = document.getElementById("invoice-print");
    if (!invoice) return;

    const canvas = await html2canvas(invoice, {
      scale: 2,
      useCORS: true,
    });

    const imgData = canvas.toDataURL("image/png");

    const pdf = new jsPDF("p", "mm", "a4");
    const pdfWidth = 210;
    const pdfHeight = 297;
    const imgHeight = (canvas.height * pdfWidth) / canvas.width;
    const finalHeight = imgHeight > pdfHeight ? pdfHeight : imgHeight;

    pdf.addImage(imgData, "PNG", 0, 0, pdfWidth, finalHeight);
    pdf.save(`Invoice-${order.id}.pdf`);
  };

  // Check if order has multiple items
  const hasMultipleItems = order.items && order.items.length > 0;
  const items = hasMultipleItems ? order.items : [{
    pages: order.pages || 0,
    copies: order.copies || 1,
    paperSize: order.paperSize || "A4",
    printColor: order.printColor || "B&W",
    bindingType: order.bindingType || "None"
  }];

  // Get customer details from order data
  const customerName = order.customer?.name || "Customer";
  const customerPhone = order.customer?.phone || "N/A";
  const customerAddress = order.customer?.address || "";
  const customerCity = order.customer?.city || "";
  const customerPincode = order.customer?.pincode || "";
  const customerState = order.customer?.state || "";
  const customerGstin = order.customer?.gstin || "";
```

```
const fullAddress = customerAddress ?
  `${customerAddress}, ${customerCity} - ${customerPincode}, ${customerState}` :
  "Address not provided";

const isCourier = order.deliveryType === 'Courier' || order.deliveryType === 'courier';
const paymentMode = order.paymentMode || 'cod';
const paymentStatus = order.paymentStatus || 'pending';

// Get payment mode display details
const getPaymentDetails = () => {
  switch (paymentMode) {
    case 'upi':
      return {
        icon: Wallet,
        title: 'UPI Payment',
        description: 'Paid via UPI',
        transactionId: order.razorpayPaymentId,
        showBankDetails: false,
        bankDetails: null
      };
    case 'card':
      return {
        icon: CreditCard,
        title: 'Card Payment',
        description: 'Paid via Credit/Debit Card',
        transactionId: order.razorpayPaymentId,
        showBankDetails: false,
        bankDetails: null
      };
    case 'bank':
      return {
        icon: FileText,
        title: 'Bank Transfer',
        description: 'Payment via Bank Transfer',
        transactionId: order.razorpayPaymentId,
        showBankDetails: true,
        bankDetails: {
          bank: 'State Bank Of India, Chandervardai, Ajmer',
          accountNo: '39918178182',
          ifsc: 'SBIN0032089',
          holder: 'Shree Education And Publication Private Limited'
        }
      };
    default:
      return {
        icon: Package,
        title: 'Cash on Delivery',
        description: 'Payment to be made at the time of delivery/pickup',
        transactionId: null,
        showBankDetails: false,
        bankDetails: null
      };
  }
};

const paymentDetails = getPaymentDetails();
const PaymentIcon = paymentDetails.icon;

return (
```

```
    <div className="fixed inset-0 bg-black/60 z-50 flex items-center justify-center p-4
animate-fade-in">
      <div className="bg-white rounded-2xl shadow-2xl max-w-4xl w-full max-h-[90vh]
overflow-y-auto animate-slide-up">
        {/* Invoice Content */}
        <div id="invoice-print" className="p-8 md:p-10 bg-white w-[850px] mx-auto">
          {/* Header */}
<div className="flex justify-between items-start border-b-2 border-secondary pb-6 mb-6">
  <div>
    <h1 className="text-xl font-black text-secondary">
      Shree Education and Publication private limited
    </h1>
    <p className="text-muted-foreground text-sm mt-1">
      Mother’s School Campus
      Gaddi Maliyan, Jonsganj Road
    </p>
    <p className="text-muted-foreground text-sm">
      Ajmer, Rajasthan – 305001
      India
    </p>
    <p className="text-muted-foreground text-sm mt-1">
      Phone: 7230001405 | Email: shreedupub@gmail.com
    </p>
    <p className="text-muted-foreground text-sm font-medium mt-1">
      GSTIN: 08ABECS6515Q1ZP
    </p>
  </div>
  <div className="text-right">
    <div className="bg-primary text-white px-5 py-3 rounded-lg inline-block max-w-full">
      <p className="text-xs opacity-90 whitespace-nowrap">TAX INVOICE</p>
      <p className="font-bold text-lg whitespace-nowrap">
        #{order.id}
      </p>
    </div>
    <p className="text-muted-foreground text-sm mt-3 whitespace-nowrap">
      Date: {order.date}
    </p>
    {isCourier ? (
      <p className="text-muted-foreground text-sm whitespace-nowrap">
        Place of Supply: {customerState || '08-Rajasthan'}
      </p>
    ) : (
      <p className="text-muted-foreground text-sm whitespace-nowrap">
        Place of Supply: Store Pickup
      </p>
    )}
  </div>
</div>

          {/* Bill To / Ship To */}
          <div className="grid grid-cols-1 md:grid-cols-3 gap-6 mb-8">
            <div className="md:col-span-2">
              <p className="text-xs font-bold text-muted-foreground uppercase tracking-wide
mb-2">
                {isCourier ? 'Bill To / Ship To' : 'Bill To'}
              </p>
              <p className="font-semibold text-lg">{customerName}</p>
              {isCourier ? (
                <>
```

```
                <p className="text-muted-foreground text-sm mt-1">{fullAddress}</p>
                <p className="text-muted-foreground text-sm">Contact No: {customerPhone}</p>
              </>
            ) : (
              <>
                <p className="text-muted-foreground text-sm">Contact No: {customerPhone}</p>
                <p className="text-muted-foreground text-sm mt-2 font-medium">
                  Pickup Location: Mother’s School Campus, Gaddi Maliyan, Jonsganj Road, Ajmer,
Rajasthan
                </p>
              </>
            )}
            {customerGstin && (
              <p className="text-muted-foreground text-sm font-medium mt-2">
                GSTIN: {customerGstin}
              </p>
            )}
            {isCourier ? (
              <p className="text-muted-foreground text-sm">State: {customerState ||
'08-Rajasthan'}</p>
            ) : (
              <p className="text-muted-foreground text-sm">State: Rajasthan</p>
            )}
          </div>

          <div>
            <p className="text-xs font-bold text-muted-foreground uppercase tracking-wide
mb-2">
              Invoice Details
            </p>
            <p className="text-sm text-muted-foreground">Invoice No: {order.id}</p>
            <p className="text-sm text-muted-foreground">Date: {order.date}</p>
            <p className="text-sm text-muted-foreground">Delivery: {order.deliveryType}</p>
            <p className="text-sm text-muted-foreground mt-2">
              Payment Status:
              <span className={`ml-2 px-2 py-0.5 rounded-full text-xs font-medium ${
                paymentStatus === 'paid' ? 'bg-green-100 text-green-700' : 'bg-yellow-100
text-yellow-700'
              }`}>
                {paymentStatus === 'paid' ? 'Paid' : 'Pending'}
              </span>
            </p>
            {paymentDetails.transactionId && (
              <p className="text-sm text-muted-foreground mt-1">
                Transaction ID: {paymentDetails.transactionId}
              </p>
            )}
          </div>
        </div>

        {/* Items Table */}
        <table className="w-full mb-8 text-sm">
          <thead>
            <tr className="bg-secondary text-white">
              <th className="p-3 text-left rounded-tl-lg">#</th>
              <th className="p-3 text-left">Item name</th>
              <th className="p-3 text-center">Pages</th>
              <th className="p-3 text-center">Copies</th>
              <th className="p-3 text-center">Paper Size</th>
```

```
              <th className="p-3 text-center">Print Color</th>
              <th className="p-3 text-right">Binding</th>
              <th className="p-3 text-right rounded-tr-lg">Amount</th>
            </tr>
          </thead>
          <tbody>
            {items.map((item, index) => {
              const itemAmount = order.amount / items.length;

              return (
                <tr key={index} className="border-b hover:bg-gray-50">
                  <td className="p-3">{index + 1}</td>
                  <td className="p-3">
                    <p className="font-medium">{item.fileName || `Item ${index + 1}`}</p>
                    <p className="text-xs text-muted-foreground">
                      {item.bindingType} binding
                    </p>
                  </td>
                  <td className="p-3 text-center">{item.pages}</td>
                  <td className="p-3 text-center">{item.copies}</td>
                  <td className="p-3 text-center">{item.paperSize}</td>
                  <td className="p-3 text-center">{item.printColor}</td>
                  <td className="p-3 text-right">{item.bindingType}</td>
                  <td className="p-3 text-right font-medium">₹{itemAmount.toFixed(2)}</td>
                </tr>
              );
            })}
          </tbody>
        </table>

        {/* Amount in Words & Totals */}
        <div className="flex flex-col md:flex-row justify-between items-start md:items-center
gap-6 mb-8">
          <div>
            <p className="font-medium">Invoice Amount In Words:</p>
            <p className="text-lg font-semibold text-secondary mt-1">
              {numberToWords(order.amount)}
            </p>
          </div>

          <div className="w-64 space-y-2 text-right">
            <div className="flex justify-between">
              <span className="text-muted-foreground">Sub Total</span>
              <span>₹{baseAmount.toFixed(2)}</span>
            </div>
            <div className="flex justify-between">
              <span className="text-muted-foreground">CGST (2.5%)</span>
              <span>₹{cgst.toFixed(2)}</span>
            </div>
            <div className="flex justify-between">
              <span className="text-muted-foreground">SGST (2.5%)</span>
              <span>₹{sgst.toFixed(2)}</span>
            </div>
            <div className="flex justify-between font-black text-lg border-t-2 border-secondary
pt-2 mt-2">
              <span>TOTAL</span>
              <span className="text-primary">₹{order.amount.toFixed(2)}</span>
            </div>
          </div>
```

```
          </div>

          {/* Payment Details */}
          <div className="grid md:grid-cols-2 gap-8 mb-8">
            <div>
              <p className="font-medium mb-2">Payment Details</p>
              <div className="flex items-center gap-2">
                <PaymentIcon className="h-5 w-5 text-primary" />
                <p className="text-muted-foreground font-medium">
                  {paymentDetails.title}
                </p>
              </div>
              <p className="text-sm text-muted-foreground mt-1">
                {paymentDetails.description}
              </p>
              {paymentStatus === 'pending' && paymentMode === 'cod' && (
                <p className="text-sm text-amber-600 mt-1">
                  * Payment to be made at the time of {isCourier ? 'delivery' : 'pickup'}
                </p>
              )}
            </div>

            {paymentDetails.showBankDetails && paymentDetails.bankDetails && (
              <div>
                <p className="font-medium mb-2">Bank Details</p>
                <p className="text-sm text-muted-foreground">
                  <strong>Bank:</strong> {paymentDetails.bankDetails.bank}
                </p>
                <p className="text-sm text-muted-foreground">
                  <strong>Account No:</strong> {paymentDetails.bankDetails.accountNo}
                </p>
                <p className="text-sm text-muted-foreground">
                  <strong>IFSC Code:</strong> {paymentDetails.bankDetails.ifsc}
                </p>
                <p className="text-sm text-muted-foreground">
                  <strong>Account Holder:</strong> {paymentDetails.bankDetails.holder}
                </p>
              </div>
            )}
          </div>

          {/* Terms & Authorized Signatory */}
          <div className="grid md:grid-cols-2 gap-8 mb-8">
            <div>
              <p className="font-medium mb-2">Terms and conditions</p>
              <p className="text-sm text-muted-foreground">
                Thank you for doing business with us. For any queries, please contact our support
team.
              </p>
              <p className="text-sm text-muted-foreground mt-1">
                Delivery: {isCourier ? 'Courier delivery within 3-5 business days' : 'Store
pickup available during business hours'}
              </p>
            </div>

            <div className="text-right">
              <p className="font-medium mb-2">For: Shree Education and Publication private
limited</p>
              <div className="mt-10 border-t border-gray-400 w-48 ml-auto pt-2">
```

```
              <p className="text-sm">Authorized Signatory</p>
            </div>
          </div>
        </div>

        {/* Acknowledgment */}
        <div className="border-t-2 border-secondary pt-6 mt-10 text-center">
          <h4 className="font-bold text-lg mb-4">Acknowledgment</h4>
          <p className="text-muted-foreground">
            Shree Education and Publication private limited
          </p>
          <div className="mt-8 grid md:grid-cols-2 gap-6 text-left text-sm">
            <div>
              <p><strong>Invoice To:</strong> {customerName}</p>
              {isCourier ? (
                <p>{fullAddress}</p>
              ) : (
                <p>Store Pickup - {customerName}</p>
              )}
              <p>Phone: {customerPhone}</p>
            </div>
            <div>
              <p><strong>Invoice Details:</strong></p>
              <p>Invoice No: {order.id}</p>
              <p>Invoice Date: {order.date}</p>
              <p>Payment Mode: {paymentDetails.title.toUpperCase()}</p>
              <p>Payment Status: {paymentStatus === 'paid' ? 'PAID' : 'PENDING'}</p>
              <p>Total: ₹{order.amount.toFixed(2)}</p>
            </div>
          </div>
          <div className="mt-10 border-t border-dashed pt-6">
            <p className="text-muted-foreground text-sm">
              Receiver's Seal & Sign .....................................................
            </p>
          </div>
        </div>
      </div>

      {/* Action Buttons */}
      <div className="px-8 pb-8 flex gap-4 no-print">
        <button
          onClick={onClose}
          className="flex-1 border border-border text-foreground font-medium py-3 rounded-lg
hover:bg-muted transition-all"
        >
          Close
        </button>
        <button
          onClick={handleDownload}
          className="flex-1 bg-primary text-primary-foreground font-bold py-3 rounded-lg
hover:bg-primary/90 transition-all flex items-center justify-center gap-2"
        >
          <Download className="h-4 w-4" /> Print / Download
        </button>
      </div>
    </div>
  </div>
);
}
```

```jsx
import { useState, useEffect } from 'react';
import { useAuth } from "@/context/AuthContext";
import { Link, useLocation, useNavigate } from 'react-router-dom';
import { Menu, X, Phone, ShoppingCart } from 'lucide-react';
import { toast } from "sonner";
import logo from '@/assets/logo2.png';
import axios from 'axios';


const navLinks = [
  { href: '/', label: 'Home' },
  { href: '/order', label: 'Place Order' },
  { href: '/tracking', label: 'Track Order' },
  { href: '/history', label: 'Order History' },
];

const API = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'https://bookprinters.in/api/api',
});

API.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default function Navbar() {
  const [isMenuOpen, setIsMenuOpen] = useState(false);
  const [isScrolled, setIsScrolled] = useState(false);
  const [cartItemCount, setCartItemCount] = useState(0);

  const location = useLocation();
  const navigate = useNavigate();

  const { isAuthenticated, logout } = useAuth();

  // ☑ Fetch cart count safely
  const fetchCartCount = async () => {
    const token = localStorage.getItem('token');

    if (!token || !isAuthenticated) {
      setCartItemCount(0);
      return;
    }

    try {
      const response = await API.get('/cart');
      const itemCount = response.data?.items?.length || 0;
      setCartItemCount(itemCount);
    } catch (error) {
      console.error("Error fetching cart count:", error);
      setCartItemCount(0);
    }
  };

  const isHomePage = location.pathname === "/";
```

```
  useEffect(() => {
    const handleScroll = () => setIsScrolled(window.scrollY > 20);
    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  useEffect(() => {
    fetchCartCount();
  }, [isAuthenticated, location.pathname]);

  useEffect(() => {
    setIsMenuOpen(false);
  }, [location]);

  // ☑ FIXED LOGOUT
  const handleLogout = () => {
    logout();
    setIsMenuOpen(false);
    setCartItemCount(0);
    toast.success("Logged out successfully");
    navigate("/");
  };

  // ☑ FIXED CART NAVIGATION (IMPORTANT FIX)
  const handleCartClick = () => {
    setIsMenuOpen(false);

    if (!isAuthenticated) {
      toast.error("Please login to view cart");
      navigate("/auth");
      return;
    }

    navigate("/cart");
  };

  return (
    <nav className={`fixed top-0 left-0 right-0 z-50 transition-all duration-300 ${
      isScrolled ? 'bg-white shadow-lg py-2 lg:py-3.5' : 'bg-white shadow-sm py-3 lg:py-5'
    }`}>
{/* <nav className={`fixed top-12 left-0 right-0 z-40 transition-all duration-300 ${
  isScrolled ? 'bg-white shadow-lg py-2 lg:py-3.5' : 'bg-white shadow-sm py-3 lg:py-5'
}`}> */}




      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center justify-between h-14">

          {/* Logo */}
          <Link to="/" className="flex items-center">
            <img src={logo} alt="Logo" className="h-14 sm:h-16 lg:h-20 w-auto" />
          </Link>

          {/* Desktop Nav */}
          <div className="hidden md:flex items-center gap-1">
```

```
            {navLinks.map((link) => (
              <Link
                key={link.href}
                to={link.href}
                className={`px-4 py-2 rounded-md text-sm font-medium ${
                  location.pathname === link.href
                    ? 'bg-primary text-primary-foreground'
                    : 'text-gray-800 hover:text-primary hover:bg-gray-100'
                }`}
              >
                {link.label}
              </Link>
            ))}
          </div>

          {/* CTA */}
          <div className="hidden md:flex items-center gap-4">

            <Link
              to="/contact"
              className="flex items-center gap-2 text-gray-700 hover:text-primary text-sm
font-medium transition-colors duration-200 hover:bg-gray-100 px-4 py-2 rounded-md"
            >
              <Phone className="h-4 w-4" />
              Contact
            </Link>

            {!isAuthenticated ? (
              <Link to="/auth" className="px-4 py-2 rounded-md text-sm font-medium text-gray-700
hover:text-primary hover:bg-gray-100 transition-all duration-200">Login</Link>
            ) : (
              <button onClick={handleLogout}  className="px-4 py-2 rounded-md text-sm font-medium
text-red-600 hover:bg-red-50 transition-all duration-200">
                Logout
              </button>
            )}

            <Link
              to="/order"
              className="px-5 py-2 rounded-md text-sm font-bold bg-primary
text-primary-foreground hover:bg-primary/90 transition-all duration-200 hover:scale-105
active:scale-95"
            >
              Order Now
            </Link>

            {/* ☑ FIXED CART BUTTON */}
            <button onClick={handleCartClick} className="relative ml-4 h-6 w-6 text-gray-700
transition-all duration-300
  hover:text-secondary hover:scale-110 active:scale-95
  drop-shadow-sm hover:drop-shadow-md cursor-pointer">
             <ShoppingCart
  className=""
/>
              {cartItemCount > 0 && (
                <span className="absolute -top-2 -right-2 bg-primary text-white text-xs
rounded-full px-1.5  ">
                  {cartItemCount > 99 ? "99+" : cartItemCount}
                </span>
```

```
            )}
          </button>

        </div>

        {/* Mobile Menu Button */}
        <button
          onClick={() => setIsMenuOpen(!isMenuOpen)}
          className="md:hidden p-2"
        >
          {isMenuOpen ? <X /> : <Menu />}
        </button>
      </div>
    </div>

    {/* Mobile Menu */}
    <div className={`md:hidden overflow-hidden transition-all duration-300 ${
      isMenuOpen ? 'max-h-[600px] opacity-100' : 'max-h-0 opacity-0'
    }`}>

      <div className="bg-white border-t px-4 py-5 space-y-2">

        {navLinks.map((link) => (
          <Link key={link.href} to={link.href} className="block py-2">
            {link.label}
          </Link>
        ))}

        {/* Mobile Cart FIX */}
        <button
          onClick={handleCartClick}
          className="flex justify-between w-full py-3"
        >
          <span>Cart</span>
          {cartItemCount > 0 && (
            <span className="bg-primary text-white text-xs px-2 rounded-full">
              {cartItemCount}
            </span>
          )}
        </button>

        {!isAuthenticated ? (
          <Link to="/auth">Login</Link>
        ) : (
          <button onClick={handleLogout} className="text-red-600">
            Logout
          </button>
        )}

      </div>
    </div>

  </nav>
 );
}

// src/store/cartStore.ts
import { create } from "zustand";
import { persist, PersistOptions, createJSONStorage } from "zustand/middleware";
```

```
import {
  fetchCart,
  saveCartToServer,
  removeCartItemFromServer,
  updateCartItemQuantityOnServer,
  clearCartOnServer
} from "../api/cartApi";
import { CartData } from "../types/cart";

interface CartState {
  cart: CartData | null;
  loading: boolean;
  error: string | null;

  // Actions
  loadCart: () => Promise<void>;
  saveCart: (cartData: CartData) => Promise<CartData>;
  updateQuantity: (itemId: string, copies: number) => Promise<CartData>;
  removeItem: (itemId: string) => Promise<CartData>;
  clearCart: () => Promise<void>;
  getItemCount: () => number;
  getTotal: () => number;
  getPrintingCost: () => number;
  getGst: () => number;
}

type CartPersist = PersistOptions<CartState, Pick<CartState, 'cart'>>;

export const useCartStore = create<CartState>()(
  persist(
    (set, get) => ({
      cart: null,
      loading: false,
      error: null,

      loadCart: async () => {
        set({ loading: true, error: null });
        try {
          const cart = await fetchCart();
          set({ cart, loading: false });
        } catch (error: any) {
          set({
            error: error.response?.data?.error || error.message,
            loading: false
          });
          console.error("Failed to load cart:", error);
        }
      },

      saveCart: async (cartData: CartData) => {
        set({ loading: true, error: null });
        try {
          const updatedCart = await saveCartToServer(cartData);
          set({ cart: updatedCart, loading: false });
          return updatedCart;
        } catch (error: any) {
          set({
            error: error.response?.data?.error || error.message,
            loading: false
```

```
    });
    throw error;
  }
},

updateQuantity: async (itemId: string, copies: number) => {
  set({ loading: true, error: null });
  try {
    const updatedCart = await updateCartItemQuantityOnServer(itemId, copies);
    set({ cart: updatedCart, loading: false });
    return updatedCart;
  } catch (error: any) {
    set({
      error: error.response?.data?.error || error.message,
      loading: false
    });
    throw error;
  }
},

removeItem: async (itemId: string) => {
  set({ loading: true, error: null });
  try {
    const updatedCart = await removeCartItemFromServer(itemId);
    set({ cart: updatedCart, loading: false });
    return updatedCart;
  } catch (error: any) {
    set({
      error: error.response?.data?.error || error.message,
      loading: false
    });
    throw error;
  }
},

clearCart: async () => {
  set({ loading: true, error: null });
  try {
    await clearCartOnServer();
    set({ cart: null, loading: false });
  } catch (error: any) {
    set({
      error: error.response?.data?.error || error.message,
      loading: false
    });
    throw error;
  }
},

getItemCount: () => {
  const { cart } = get();
  return cart?.items?.length || 0;
},

getTotal: () => {
  const { cart } = get();
  return cart?.totals?.totalWithDelivery || 0;
},
```

```
      getPrintingCost: () => {
        const { cart } = get();
        return cart?.totals?.printingCost || 0;
      },

      getGst: () => {
        const { cart } = get();
        return cart?.totals?.gst || 0;
      },
    }),
    {
      name: "cart-storage",
      storage: createJSONStorage(() => localStorage),
      partialize: (state) => ({ cart: state.cart }),
    } as CartPersist
  )
);
```